Tutorial 2: MLP Classifier

Name: Laiba Niazi

Roll Number: 536672

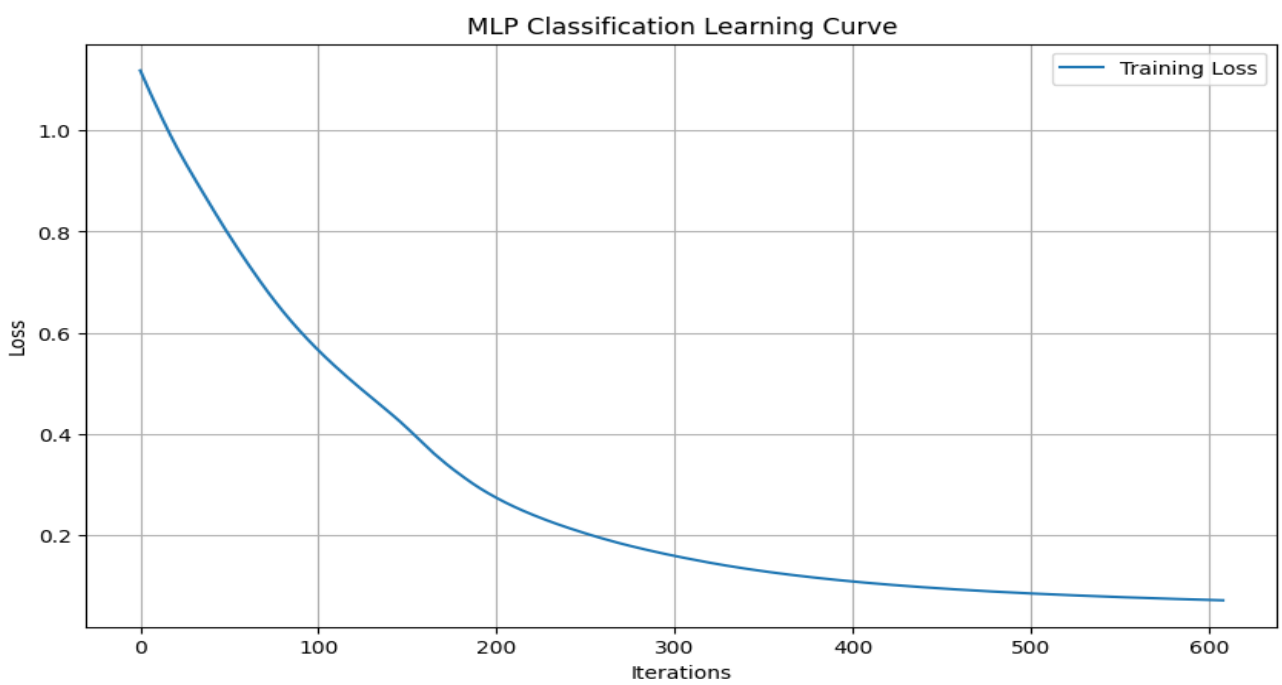
1. Introduction

The objective of this experiment was to implement a Multi-Layer Perceptron (MLP) neural network to classify Iris flower species. The study focused on how changing the internal architecture (layers/neurons) and the optimization parameters (learning rate) affects the model's accuracy and speed of convergence.

2. Baseline Model Performance

The initial model was configured with two hidden layers of 10 neurons each (10, 10), a learning rate of 0.001, and a relu activation function.

- **Final Accuracy:** 1.00 (100%)
- **Iterations to Converge:** 609
- **Output Activation:** Softmax



```python
y_pred = mlp.predict(X_test_scaled)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

print("Classification Report:", classification_report(y_test, y_pred))
```

✓ 0.0s

```
Accuracy: 1.00
Classification Report:              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

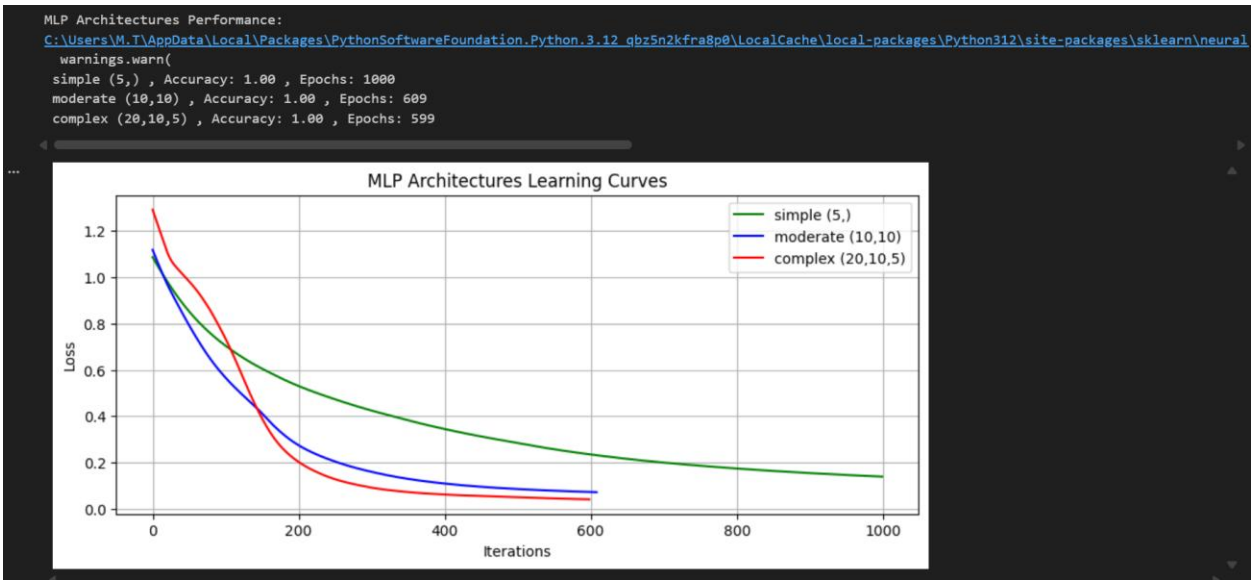
3. Task 1: Impact of Hidden Layer Architecture

In this task, we compared three different network depths to observe how complexity influences learning.

Architecture	Label	Neurons per Layer	Accuracy	Epochs to Converge
Simple	(5,)	One layer of 5	1.00	1000 (Max reached)
Moderate	(10, 10)	Two layers of 10	1.00	609
Complex	(20, 10, 5)	Three decreasing layers	1.00	599

Observations:

- The **Simple** model failed to converge within the default iteration limit, suggesting that 5 neurons might be slightly too few to find the optimal global minimum quickly.
- The **Complex** model converged the fastest (599 iterations), showing that for this dataset, a slightly deeper network finds a solution in fewer steps.



4. Task 2: Impact of Learning Rate

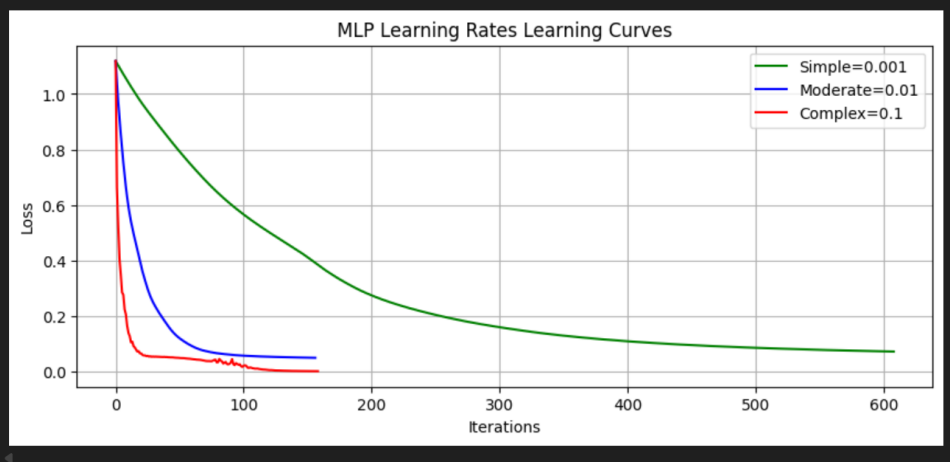
The learning rate determines the "step size" the model takes during optimization. We tested three values: 0.001, 0.01, and 0.1.

Learning Rate	Accuracy	Convergence Speed	Notes
0.001 (Default)	1.00	609 iterations	Smooth, stable descent.
0.01 (Moderate)	1.00	157 iterations	Significantly faster; no loss in accuracy.
0.1 (High)	0.96	159 iterations	Fast, but accuracy dropped slightly.

Observations:

- Increasing the learning rate to 0.01 was the "sweet spot," resulting in 4x faster training while maintaining 100% accuracy.
- The high learning rate (0.1) caused the model to "overshoot" the optimal weights, leading to a lower final accuracy of 96%.

```
MLP Learning Rates Performance:
 Simple=0.001 , Accuracy: 1.00 , Epochs: 609
 Moderate=0.01 , Accuracy: 1.00 , Epochs: 157
 Complex=0.1 , Accuracy: 0.96 , Epochs: 159
```



5. Conclusion

- **Feature Scaling:** The use of StandardScaler was critical for the MLP to process all features on the same scale.
- **Complexity:** For simple datasets like Iris, extremely deep networks are not necessary, but a "Simple" architecture with very few neurons may struggle to converge.
- **Learning Rate:** The learning rate is the most sensitive hyperparameter. While higher rates speed up training, they risk destabilizing the model and reducing final accuracy.